

4 - Web

Name of report: CIA_UNIX_LAB_4_Nikita_Niakhai

Course: Unix-based Internet Applications

Performed by Nikita Niakhai

Date submission: 12.12.2025

Task 1 - Install & Configure Virtual Hosts

1. Download, verify, build and install the webserver daemon from the source

- Installed dependencies:

```
sudo apt update  
sudo apt install -y build-essential libpcre3 libpcre3-dev zlib1g zlib1g-dev \  
libssl-dev libmaxminddb0 libmaxminddb-dev mmdb-bin wget git tar ca-certificates
```

- Packages installed:

- build-essential → gcc, g++, make, etc.
- libpcre3-dev → PCRE library for regex support
- zlib1g-dev → gzip compression support
- libssl-dev → OpenSSL headers for HTTPS
- libmaxminddb-dev → GeoIP2 library
- mmdb-bin → mmdblookup utility
- wget, git, tar, ca-certificates → utilities

- Downloaded latest stable source of web-server:

```
cd /usr/local/src  
sudo wget https://nginx.org/download/nginx-1.26.3.tar.gz  
sudo wget https://nginx.org/download/nginx-1.26.3.tar.gz.asc
```

- Extracted: tar -zxvf nginx-1.26.0.tar.gz

- Configured nginx

```
sudo tar -xzf nginx-1.26.3.tar.gz
cd nginx-1.26.3
sudo ./configure \
    --prefix=/etc/nginx \
    --sbin-path=/usr/sbin/nginx \
    --conf-path=/etc/nginx/nginx.conf \
    --error-log-path=/var/log/nginx/error.log \
    --http-log-path=/var/log/nginx/access.log \
    --pid-path=/var/run/nginx.pid \
    --lock-path=/var/run/nginx.lock \
    --with-http_ssl_module \
    --with-http_v2_module \
    --with-http_realip_module \
    --with-http_stub_status_module \
    --with-threads \
    --with-http_gzip_static_module \
    --add-module=/usr/local/src/nginx_http_geoip2_module
```

- Flags explained:
 - -prefix=/etc/nginx → base directory for configuration files
 - -sbin-path=/usr/sbin/nginx → location of the nginx binary
 - -with-http_ssl_module → enables HTTPS support
 - -with-http_v2_module → HTTP/2
 - -with-http_stub_status_module → /status page
 - -with-threads → thread pool support
 - -with-http_gzip_static_module → serve pre-compressed files
 - -add-module= → statically compiles the GeoIP2 module
- Build & Install

```
sudo make -j$(nproc)
```

```
sudo make install
```

- `-j$(nproc)` uses all CPU cores for parallel compilation

2. Define the root directory and then two virtual hosts (and configure DNS records or wildcard accordingly)

- I configured, applied static IP address using `netplan`, opened ports for 80 and 443 ports

```
(nikita@nikita)-[~/Desktop]
$ cat /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: true
    eth1:
      addresses: [192.168.56.101/24]
      routes:
        - to: default
          via: 192.168.56.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
```

Figure. `netplan` config

- Verified ip address, using `ip a`

```
(nikita@nikita)-[~/Desktop]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state up group default qlen 1000
    link/ether 08:00:27:9a:fb:6a brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic eth0
        valid_lft 86111sec preferred_lft 86111sec
    inet6 fd17:625c:f037:2:d52b:4fef:a11a:51ed/64 scope global temporary
        valid_lft 86363sec preferred_lft 14363sec
    inet6 fd17:625c:f037:2:a00:27ff:fe9a:fb6a/64 scope global dynamic mcast
        valid_lft 86363sec preferred_lft 14363sec
    inet6 fe80::a00:27ff:fe9a:fb6a/64 scope link proto kernel lladdr
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state up group default qlen 1000
    link/ether 08:00:27:d8:66:5e brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fed8:665e/64 scope link proto kernel lladdr
        valid_lft forever preferred_lft forever
```

Figure. interfaces enabled and addresses on VM

- Setup DNS records inside hosts interface in my Windows host in order to access web pages.

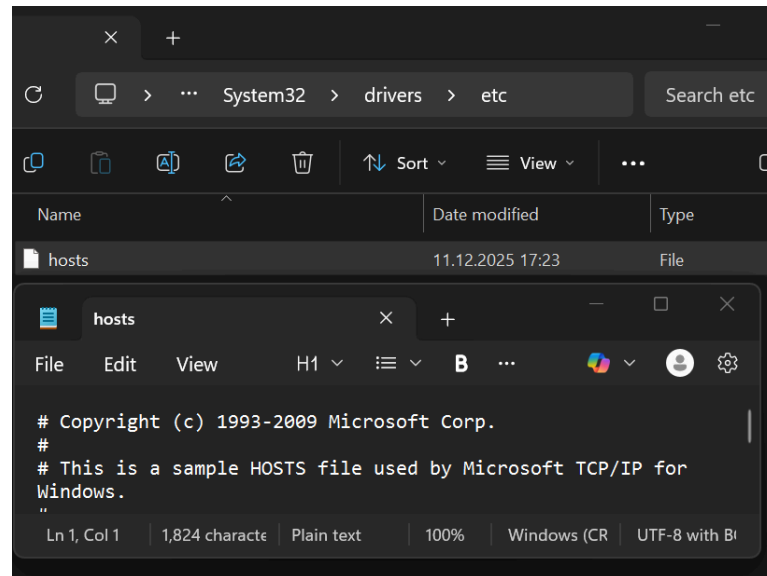


Figure. `hosts` content

- Created root directories, files for simple html pages, set permissions and access policy

```
sudo mkdir -p /var/www/aaa.st99.sne22.ru/html
sudo mkdir -p /var/www/bbb.st99.sne22.ru/html

sudo chown -R nikita:nikita /var/www/aaa.st99.sne22.ru/html
sudo chown -R nikita:nikita /var/www/bbb.st99.sne22.ru/html
sudo chmod -R 755 /var/www
```

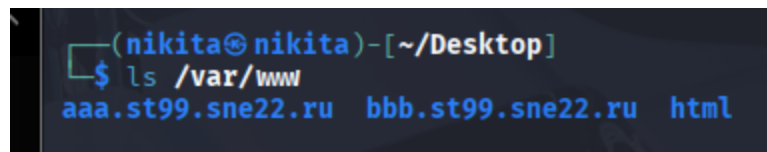


Figure. Root directories

- Created configs for the server and `nginx`:
 - `sudo mkdir -p /etc/nginx/sites-available /etc/nginx/sites-enabled`
 - Created configs for the servers there

```
GNU nano 8.4 /etc/nginx/sites-available/aaa.conf *
server {
    listen 80;
    listen [::]:80;
    server_name aaa.st99.sne22.ru;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name aaa.st99.sne22.ru;

    ssl_certificate /root/ca/certs/aaa.st99.sne22.ru.cert.pem;
    ssl_certificate_key /root/ca/private/aaa.st99.sne22.ru.key.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;
    ssl_session_tickets off;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" >

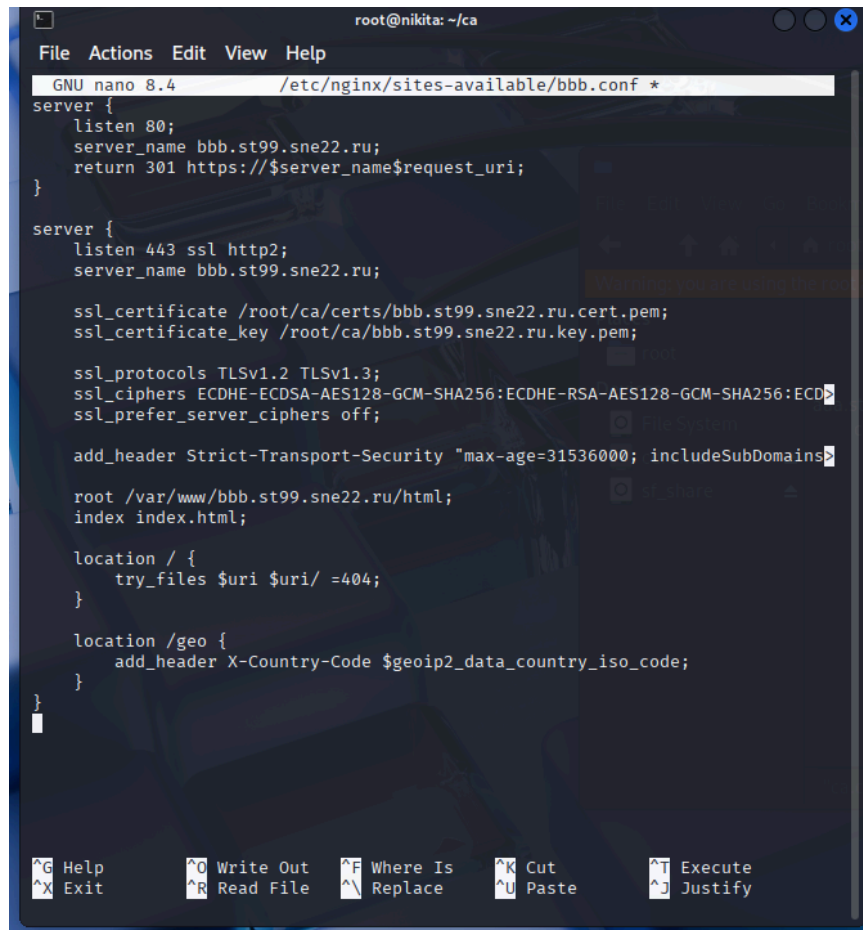
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    root /var/www/aaa.st99.sne22.ru/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /geo {
        add_header X-Country-Code $geoip2_data_country_iso_code;
    }
}
```

Figure. `/etc/nginx/sites-available/aaa.conf`



```
root@nikita: ~/ca
File Actions Edit View Help
GNU nano 8.4 /etc/nginx/sites-available/bbb.conf *
server {
    listen 80;
    server_name bbb.st99.sne22.ru;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name bbb.st99.sne22.ru;

    ssl_certificate /root/ca/certs/bbb.st99.sne22.ru.cert.pem;
    ssl_certificate_key /root/ca/bbb.st99.sne22.ru.key.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECD
    ssl_prefer_server_ciphers off;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains";

    root /var/www/bbb.st99.sne22.ru/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /geo {
        add_header X-Country-Code $geoip2_data_country_iso_code;
    }
}
```

Figure. `/etc/nginx/sites-available/bbb.conf` — identical except `server_name`, `root`, and certificate paths changed to `bbb`.

Security tuning explanations:

- `listen 443 ssl http2` → enables HTTP/2 over TLS
- `ssl_protocols TLSv1.2 TLSv1.3` → only modern secure protocols
- `ssl_ciphers` → Mozilla "Intermediate" compatibility list (secure and widely supported)
- `ssl_prefer_server_ciphers off` → allows client to choose strongest cipher
- `ssl_session_cache/tickets` → improves performance while disabling tickets for forward secrecy

- Strict-Transport-Security → forces HTTPS for 1 year
- Additional headers → prevent MIME sniffing, clickjacking, XSS, and control referrer leakage

```
(root@nikita)-[~/ca]
# ln -s /etc/nginx/sites-available/aaa.conf /etc/nginx/sites-enabled/

(root@nikita)-[~/ca]
# ln -s /etc/nginx/sites-available/bbb.conf /etc/nginx/sites-enabled/

(root@nikita)-[~/ca]
```

Figure. SymLinked configs from `sites-available` folder to prod folder `sites-enabled`

- Created `nginx.conf`

```
GNU nano 8.4 /etc/nginx/nginx.conf *
user www-data;
worker_processes auto;
worker_cpu_affinity auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 768;
    use epoll;
    # multi_accept on;
}

http {
    include mime.types;
    default_type application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for" '
        'country="$geoip2_data_country_iso_code"';

    access_log /var/log/nginx/access.log main;
    sendfile on;
    tcp_nopush on;
    keepalive_timeout 65;
    gzip on;
    gzip_static on;

    geoip2 /usr/share/GeoIP/GeoLite2-Country.mmdb {
        $geoip2_data_country_iso_code country iso_code;
    }

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
```

Figure. `/etc/nginx/nginx.conf`

- geoip2 → loads the country database and creates variable \$geoip2_country_iso
- include → loads all virtual hosts from sites-enabled
- Included configs for the servers in nginx.conf → `include /etc/nginx/sites-enabled/*;`
- log_format main → custom access log including GeoIP country
- tcp_nopush on; + tcp_nodelay on; → optimizes TCP packet handling

3. Create a simple, unique HTML page for each virtual host to make sure that the server can correctly serve it

- For aaa: `/var/www/aaa/public_html/index.html` .
- For bbb: `/var/www/bbb/public_html/index.html` .
- Tested access from Windows host machine

```
(nikita@nikita) - [~/Desktop]
$ cat /var/www/aaa.st99.sne22.ru/html/index.html
<!DOCTYPE html>
<html>
<head>
  <title>AAA Virtual Host</title>
</head>
<body>
  <h1>Welcome to aaa.st99.sne22.ru</h1>
  <p>This is the AAA server block.</p>
</body>
</html>
```

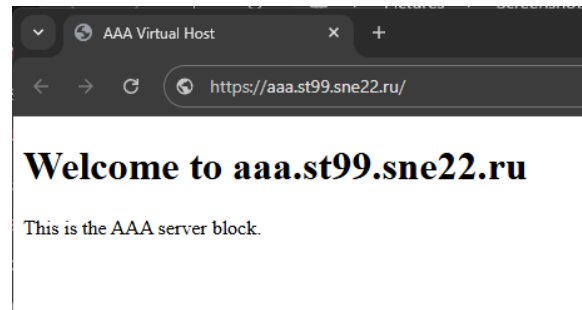


Figure. HTML page for aaa site

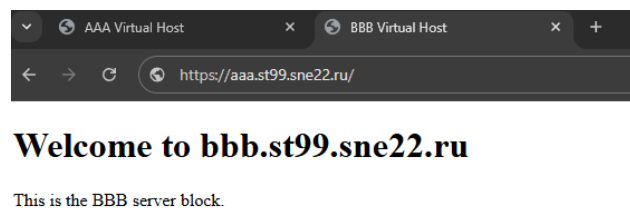


Figure. Pages accessed

4. Check the configuration syntax, start the daemon and enable it at boot time

- Check syntax: `sudo nginx -t`.
- Enable at boot: `sudo systemctl start/enable/restart nginx`

```
(root@nikita)-[~/ca]
# nginx -t
nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2"
directive instead in /etc/nginx/sites-enabled/aaa.conf:8
nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2"
directive instead in /etc/nginx/sites-enabled/bbb.conf:8
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

(root@nikita)-[~/ca]
# systemctl restart nginx

(root@nikita)-[~/ca]
# ss -tuln | grep ':80\|:443'
tcp    LISTEN 0      511          0.0.0.0:443      0.0.0.0:*
tcp    LISTEN 0      511          0.0.0.0:80       0.0.0.0:*
tcp    LISTEN 0      511          [::]:80         [::]:*
```

Figure. Output of `nginx -t` and `systemctl restart nginx` and listening ports.

5. Use curl to display the contents of a full HTTP/1.1 session served by your server

- Runed: `curl -v`

```

C:\Windows\System32>curl -I -k -v https://aaa.st99.sne22.ru -p 443
* Host aaa.st99.sne22.ru:443 was resolved.
* IPv6: (none)
* IPv4: 192.168.56.101
* Trying 192.168.56.101:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to aaa.st99.sne22.ru (192.168.56.101 port 443) from 192.168.56.1 port 48724
* using HTTP/1.x
> HEAD / HTTP/1.1
> Host: aaa.st99.sne22.ru
> User-Agent: curl/8.16.0
> Accept: */*

* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 200 OK
HTTP/1.1 200 OK
< Server: nginx/1.26.3
Server: nginx/1.26.3
< Date: Thu, 11 Dec 2025 20:27:10 GMT
Date: Thu, 11 Dec 2025 20:27:10 GMT
< Content-Type: text/html
Content-Type: text/html
< Content-Length: 180
Content-Length: 180
< Last-Modified: Thu, 11 Dec 2025 19:03:30 GMT
Last-Modified: Thu, 11 Dec 2025 19:03:30 GMT
< Connection: keep-alive
Connection: keep-alive
< ETag: "693b1582-b4"
ETag: "693b1582-b4"
< Strict-Transport-Security: max-age=31536000; includeSubDomains
Strict-Transport-Security: max-age=31536000; includeSubDomains
< Accept-Ranges: bytes
Accept-Ranges: bytes

```

Figure. Full curl -v output for HTTP session.

6. Explain the meaning of each request and reply header

- Request headers: User-Agent (client software), Host (virtual host name), Accept (accepted content types).
- Response headers: Server (web server software), Date (response time), Content-Type (response MIME type), Content-Length (body size), Connection (keep-alive or close).

Task 2 - SSL/TLS

1. Enable SSL/TLS and tune the various settings to make it as secure as possible

- Add to server blocks: `listen 443 ssl; ssl_certificate /etc/ssl/certs/certificate; ssl_certificate_key /etc/ssl/private/key; .`
- Tune: In http block, add `ssl_protocols TLSv1.2 TLSv1.3; ssl_ciphers HIGH:!aNULL:!MD5; ssl_prefer_server_ciphers on; ssl_session_cache shared:SSL:10m; ssl_session_timeout 10m; .`

- Redirect HTTP to HTTPS: Add server block `server { listen 80; server_name aaa.stX.sne22.ru; return 301 https://$host$request_uri; } .`
- Reload: `sudo nginx -s reload .`

```

root@nikita: ~
File Actions Edit View Help
GNU nano 8.4 /etc/nginx/sites-enabled/aaa.conf *
server {
    listen 80;
    server_name aaa.st99.sne22.ru;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name aaa.st99.sne22.ru;

    ssl_certificate /root/ca/certs/aaa.st99.sne22.ru.cert.pem;
    ssl_certificate_key /root/ca/aaa.st99.sne22.ru.key.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" a

    root /var/www/aaa.st99.sne22.ru/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /geo {
        add_header X-Country-Code $geoip2_data_country_iso_code;
    }
}

```

```

GNU nano 8.4 /etc/nginx/sites-enabled/bbb.conf *
server {
    listen 80;
    server_name bbb.st99.sne22.ru;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name bbb.st99.sne22.ru;

    ssl_certificate /root/ca/certs/bbb.st99.sne22.ru.cert.pem;
    ssl_certificate_key /root/ca/bbb.st99.sne22.ru.key.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" a

```

Figure. Updated server block with SSL settings.

```

C:\Windows\System32>curl -I -k https://aaa.st99.sne22.ru
HTTP/1.1 200 OK
Server: nginx/1.26.3
Date: Thu, 11 Dec 2025 20:23:34 GMT
Content-Type: text/html
Content-Length: 180
Last-Modified: Thu, 11 Dec 2025 19:03:30 GMT
Connection: keep-alive
ETag: "693b1582-b4"
Strict-Transport-Security: max-age=31536000; includeSubDomains
Accept-Ranges: bytes

```

2. Describe how you created your own certificate(s) e.g. with Let's encrypt (certbot) or self-signed and re-validate every virtual-host. Explain your security tuning process

- Modified `openssl.cnf` to direct to my keys and certs

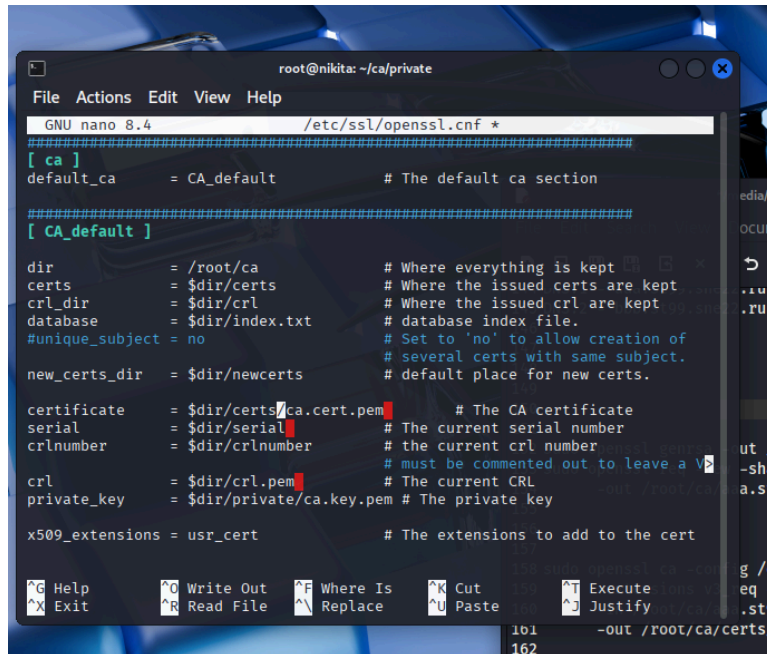


Figure. `openssl.cnf`

- Created CA directory structure

```
sudo mkdir -p /root/ca/{certs,crl,newcerts,private}
cd /root/ca
sudo chmod 700 private
touch index.txt
echo 1000 > serial
```

- Generated CA key and root certificate
 - `sudo openssl genrsa -aes256 -out private/ca.key.pem 4096`
 - `sudo chmod 400 private/ca.key.pem`

```
(root@nikita)-[~/ca]
# openssl req -config /etc/ssl/openssl.cnf -x509 -new -nodes -key private/ca.key.pem -sha256 -days 3650 -out certs/ca.cert.pem
Enter pass phrase for private/ca.key.pem:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:RU
State or Province Name (full name) [Some-State]:Innopolis
Locality Name (eg, city) []:Innopolis
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Innopolis
Organizational Unit Name (eg, section) []:Student X
Common Name (e.g. server FQDN or YOUR name) []:aaa.st99.sne22.ru
Email Address []:
```

Figure. `sudo openssl req -config /etc/ssl/openssl.cnf -x509 -new -nodes -key private/ca.key.pem -sha256 -days 3650 -out certs/ca.cert.pem`

- Server certificates (same process for both hosts) `req.cnf` and `req-bb.cnf` for SAN support

```
(root@nikita)-[~]
# cat /root/ca/req.cnf
[req]
default_bits = 2048
prompt = no
default_md = sha256
req_extensions = req_ext
distinguished_name = dn

[dn]
C = RU
ST = Innopolis
L = Innopolis
O = Innopolis
OU = Student X
CN = aaa.st99.sne22.ru

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = aaa.st99.sne22.ru

(root@nikita)-[~]
# cat /root/ca/req-bb.cnf
[req]
default_bits = 2048
prompt = no
default_md = sha256
req_extensions = req_ext
distinguished_name = dn

[dn]
C = RU
ST = Innopolis
L = Innopolis
O = Innopolis
OU = Student X
CN = bbb.st99.sne22.ru

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = bbb.st99.sne22.ru
```

Figure. Cnf files

- Generated key and CSR for aaa:

```

(root@nikita)-[~/ca]
# nano req.cnf

(root@nikita)-[~/ca]
# openssl genrsa -out /root/ca/aaa.st99.sne22.ru.key.pem 2048

(root@nikita)-[~/ca]
# openssl req -new -sha256 -key /root/ca/aaa.st99.sne22.ru.key.pem -out /root/ca/aaa.st99.sne22.ru.csr.pem -config req.cnf

```

Figure. Key and csr for aaa

- Repeated for bbb with CN = bbb.st99.sne22.ru.

```

(root@nikita)-[~/ca]
# openssl req -new -sha256 -key /root/ca/bbb.st99.sne22.ru.key.pem -out /root/ca/bbb.st99.sne22.ru.csr.pem -config req-bb.cnf

```

Figure. Key and csr for aaa

- Sign server certificates

```

sudo openssl ca -config /etc/ssl/openssl.cnf \
    -extensions v3_req -days 375 -notext -md sha256 \
    -in /root/ca/aaa.st99.sne22.ru.csr.pem \
    -out /root/ca/certs/aaa.st99.sne22.ru.cert.pem

```

```
(root@nikita)-[~/ca]
# openssl ca -config /etc/ssl/openssl.cnf -extensions v3_req -days 375 -not
ext -md sha256 -in /root/ca/aaa.st99.sne22.ru.csr.pem -out /root/ca/certs/aaa
.st99.sne22.ru.cert.pem
Using configuration from /etc/ssl/openssl.cnf
Enter pass phrase for /root/ca/private/ca.key.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4096 (0x1000)
    Validity
        Not Before: Dec 11 19:36:26 2025 GMT
        Not After : Dec 21 19:36:26 2026 GMT
    Subject:
        countryName             = RU
        stateOrProvinceName     = Innopolis
        organizationName        = Innopolis
        organizationalUnitName   = Student X
        commonName               = aaa.st99.sne22.ru
    X509v3 extensions:
        X509v3 Basic Constraints:
            CA:FALSE
        X509v3 Key Usage:
            Digital Signature, Non Repudiation, Key Encipherment
Certificate is to be certified until Dec 21 19:36:26 2026 GMT (375 days)
Sign the certificate? [y/n]:y

1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
```

Figure. Server certificates signed for aaa

- Repeated for bbb (with `req-bb.cnf`).

```
(root@nikita)-[~/ca]
# openssl ca -config /etc/ssl/openssl.cnf -extensions v3_req -days 375 -note
xt -md sha256 -in /root/ca/bbb.st99.sne22.ru.csr.pem -out /root/ca/certs/bbb.s
t99.sne22.ru.cert.pem
Using configuration from /etc/ssl/openssl.cnf
Enter pass phrase for /root/ca/private/ca.key.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4097 (0x1001)
    Validity
        Not Before: Dec 11 19:40:32 2025 GMT
        Not After : Dec 21 19:40:32 2026 GMT
    Subject:
        countryName           = RU
        stateOrProvinceName   = Innopolis
        organizationName      = Innopolis
        organizationalUnitName = Student X
        commonName            = bbb.st99.sne22.ru
    X509v3 extensions:
        X509v3 Basic Constraints:
            CA:FALSE
        X509v3 Key Usage:
            Digital Signature, Non Repudiation, Key Encipherment
Certificate is to be certified until Dec 21 19:40:32 2026 GMT (375 days)
Sign the certificate? [y/n]:y

1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
```

Figure. Server certificates signed for bbb

- Install CA into Windows host
 - Copied `/root/ca/certs/ca.cert.pem` to Windows desktop via shared folder → renamed to `lab-ca.crt`
 - Double-clicked → Install Certificate → Local Machine → Place all certificates in Trusted Root Certification Authorities

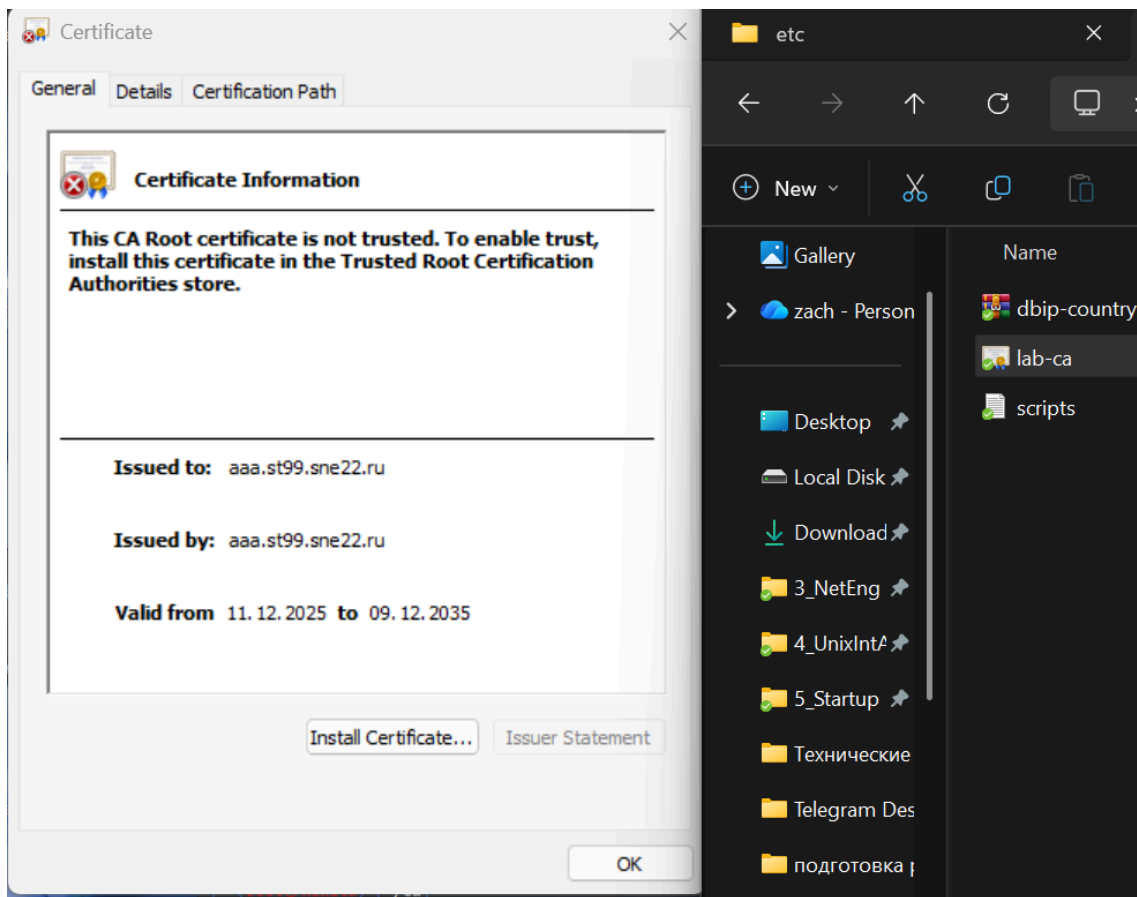


Figure. Installation of certificate into windows local machine

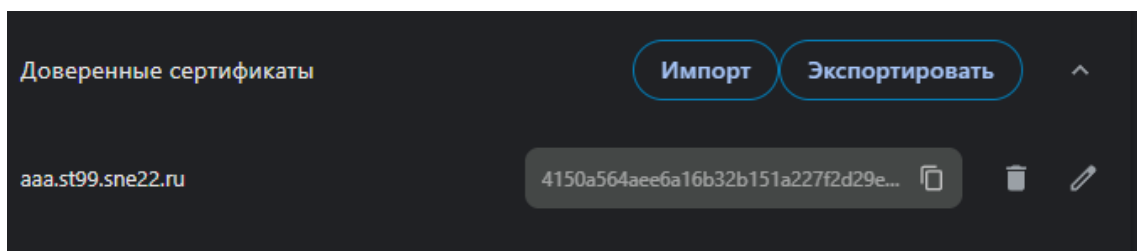


Figure. Installed certificate in a Chrome browser

- Tuning process: selected strong ciphers, enabled session caching for performance, HSTS: `add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;`

```
# Test aaa virtual host (headers only)
curl -I --cacert C:\Users\vaper\Desktop\lab-ca.crt https://aaa.st99.sne22.ru
```

U

Test bbb virtual host

```
curl -I --cacert C:\Users\vaper\Desktop\lab-ca.crt https://bbb.st99.sne22.ru
```

Full page content (aaa)

```
curl --cacert C:\Users\vaper\Desktop\lab-ca.crt https://aaa.st99.sne22.ru
```

Verbose output showing TLS handshake

```
curl -v --cacert C:\Users\vaper\Desktop\lab-ca.crt https://aaa.st99.sne22.ru
```

```
C:\Users\vaper>curl -v --cacert C:\Users\vaper\Desktop\aaa.st99.sne22.ru.crt https://aaa.st99.sne22.ru
* Host aaa.st99.sne22.ru:443 was resolved.
* IPv6: (none)
* IPv4: 192.168.56.101
* Trying 192.168.56.101:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* schannel: added 1 certificate(s) from CA file 'C:\Users\vaper\Desktop\aaa.st99.sne22.ru.crt'
* schannel: connection hostname (aaa.st99.sne22.ru) validated against certificate name (aaa.st99.sne22.ru)
* ALPN: server accepted http/1.1
* Established connection to aaa.st99.sne22.ru (192.168.56.101 port 443) from 192.168.56.1 port 36027
* using HTTP/1.x
> GET / HTTP/1.1
> Host: aaa.st99.sne22.ru
> User-Agent: curl/8.16.0
> Accept: */*
>
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: reusing certificate store from cache
* schannel: connection hostname (aaa.st99.sne22.ru) validated against certificate name (aaa.st99.sne22.ru)
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: reusing certificate store from cache
* schannel: connection hostname (aaa.st99.sne22.ru) validated against certificate name (aaa.st99.sne22.ru)
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 200 OK
< Server: nginx/1.26.3
< Date: Fri, 12 Dec 2025 12:38:47 GMT
< Content-Type: text/html
< Content-Length: 180
< Last-Modified: Thu, 11 Dec 2025 19:03:30 GMT
< Connection: keep-alive
< Vary: Accept-Encoding
< ETag: "693b1582-b4"
< Strict-Transport-Security: max-age=31536000; includeSubDomains
< X-Content-Type-Options: nosniff
< X-Frame-Options: DENY
< X-XSS-Protection: 1; mode=block
< Referrer-Policy: strict-origin-when-cross-origin
< Accept-Ranges: bytes
<
<!DOCTYPE html>
<html>
<head>
  <title>AAA Virtual Host</title>
</head>
<body>
  <h1>Welcome to aaa.st99.sne22.ru</h1>
  <p>This is the AAA server block.</p>

```

Figure. HTTPS curl test.

- **HTTPS is fully functional**
 - HTTP/1.1 200 OK
 - Connection established on port 443 to 192.168.56.101

- TLS handshake completed successfully using your local CA
- schannel validated the hostname correctly: connection hostname (aaa.st99.sne22.ru) validated against certificate name
- Security tuning is active (all headers present)

Task 3 (bonus) - GeoIP

1. Enable GeoIP on your chosen web-server (is only NGINX capable to do this?) and show how to take advantage of it with real examples

- Compile Nginx with module: Added `-with-http_geoip_module` to configure.

```
cd nginx-1.26.3
sudo ./configure \
  --prefix=/etc/nginx \
  --sbin-path=/usr/sbin/nginx \
  --conf-path=/etc/nginx/nginx.conf \
  --error-log-path=/var/log/nginx/error.log \
  --http-log-path=/var/log/nginx/access.log \
  --pid-path=/var/run/nginx.pid \
  --lock-path=/var/run/nginx.lock \
  --with-http_ssl_module \
  --with-http_v2_module \
  --with-http_realip_module \
  --with-http_stub_status_module \
  --with-threads \
  --with-http_gzip_static_module \
  --with-http_geoip_module \
  --add-module=/usr/local/src/ngx_http_geoip2_module
```

- I download databases.
 - I visited <https://db-ip.com/db/download/ip-to-country-lite>

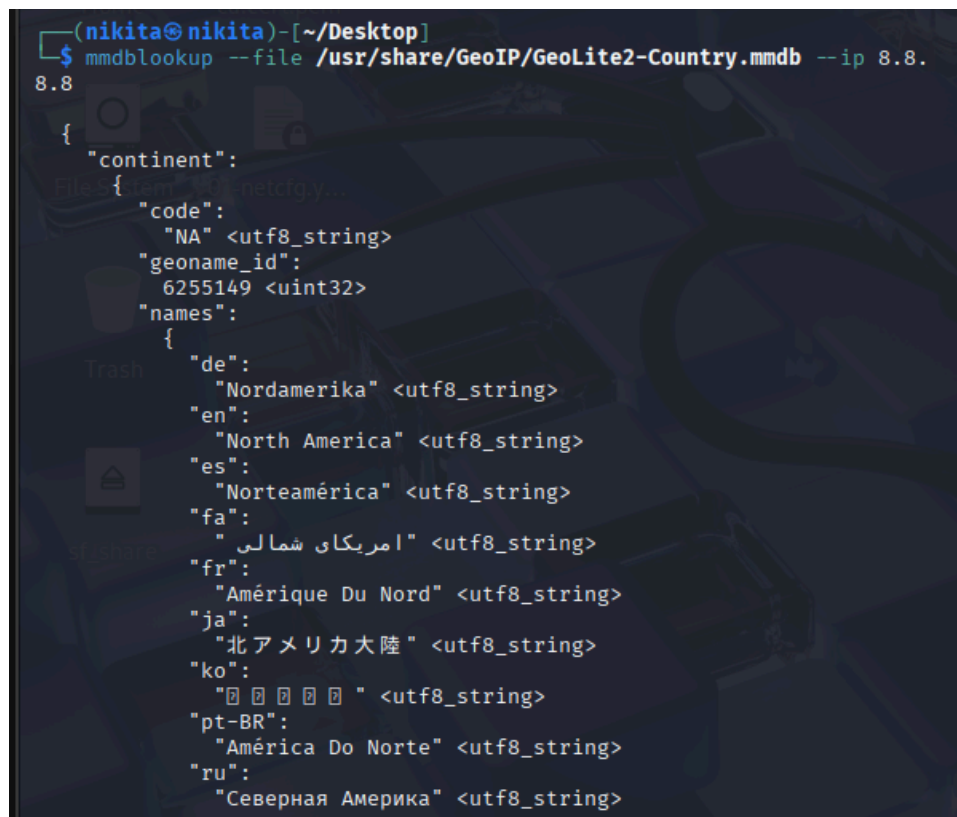
- Selected free current version for db , chose MMDB format (GeoIP2/GeoLite2 compatible)
 - `wget https://download.db-ip.com/free/dbip-country-lite-2025-12.mmdb.gz`
 - Configured database:


```
sudo mkdir -p /usr/share/GeoIP
```

```
cd /usr/share/GeoIP
```

```
sudo gunzip dbip-country-lite-2025-12.mmdb.gz
```

```
sudo mv dbip-country-lite-2025-12.mmdb GeoLite2-Country.mmdb
```
 - Verified database.
- Output showed correct country information for Google DNS → database working.



```
(nikita@nikita)-[~/Desktop]
$ mmdblookup --file /usr/share/GeoIP/GeoLite2-Country.mmdb --ip 8.8.8.8
{
  "continent": "NA",
  "code": "NA",
  "geoname_id": 6255149,
  "names": {
    "de": "Nordamerika",
    "en": "North America",
    "es": "Norteamérica",
    "fa": "امریکای شمالی",
    "fr": "Amérique Du Nord",
    "ja": "北アメリカ大陸",
    "ko": "북아메리카",
    "pt-BR": "América Do Norte",
    "ru": "Северная Америка"
  }
}
```

Figure. DNS query for Google in database

- Configured geoip in configs for nginx files (nginx.conf and aaa.conf and bbb.conf)
 - **GeoIP Directive in nginx.conf** (inside the http {} block):
 - geoip2 /usr/share/GeoIP/GeoLite2-Country.mmdb { }: Loads the country database from the specified path.
 - Inside the block: \$geoip2_country_iso \$geoip2_data_country_iso_code country iso_code;
 - Defines the variable \$geoip2_country_iso (alias for \$geoip2_data_country_iso_code).
 - Extracts the two-letter ISO country code (e.g., "US", "RU") from the database based on the client's IP address.
 - **Logging Integration:**
 - Custom log format main in nginx.conf: Includes country="\$geoip2_data_country_iso_code" at the end.
 - Access log line: access_log /var/log/nginx/access.log main;
 - Result: Each entry in /var/log/nginx/access.log appends the detected country code (e.g., country="US" or empty for private/unmatched IPs).
 - Added in virtual host server blocks: add_header X-Geo-Country \$geoip2_data_country_iso_code always;
 - Allows direct observation of the detected country in HTTP response headers (e.g., X-Geo-Country: RU).
- Tested that geoip works by assigning country code, looked in logs for specific query.

```
C:\Users\vaper>curl -v --cacert C:\Users\vaper\Desktop\aaa.st99.sne22.ru.crt -H "X-Forwarded-For: 8.8.8.8, 77.88.8.8" https://aaa.st99.sne22.ru/
* Host aaa.st99.sne22.ru:443 was resolved.
* IPv6: (none)
* IPv4: 192.168.56.101
* Trying 192.168.56.101:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* schannel: added 1 certificate(s) from CA file 'C:\Users\vaper\Desktop\aaa.st99.sne22.ru.crt'
* schannel: connection hostname (aaa.st99.sne22.ru) validated against certificate name (aaa.st99.sne22.ru)
* ALPN: server accepted http/1.1
* Established connection to aaa.st99.sne22.ru (192.168.56.101 port 443) from 192.168.56.1 port 9553
* using HTTP/1.x
> GET / HTTP/1.1
> Host: aaa.st99.sne22.ru
> User-Agent: curl/8.16.0
> Accept: */*
> X-Forwarded-For: 8.8.8.8, 77.88.8.8
>
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: reusing certificate store from cache
* schannel: connection hostname (aaa.st99.sne22.ru) validated against certificate name (aaa.st99.sne22.ru)
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: reusing certificate store from cache
* schannel: connection hostname (aaa.st99.sne22.ru) validated against certificate name (aaa.st99.sne22.ru)
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 200 OK
```

```
(root@nikita)-[~/ca/certs]
# tail -f /var/log/nginx/access.log
192.168.56.1 - - [12/Dec/2025:07:47:46 -0500] "GET / HTTP/1.1" 200 180 "-" "curl /8.16.0" "8.8.8.8, 77.88.8.8" country="RU"
```

Figure. GeoIP configuration and access log examples from hosts IP.

Summary

NGINX is not the only web server capable of handling IP geolocation functionality, such as looking up a client's country based on their IP address and using it for logging, access control, redirects, or custom headers. Web-server like Apache (third-party modules), Caddy (community plugins), LiteSpeed (built-in support, easy configuration) also provide such functionality referring to cross platforms like MaxMind and GeoIP2.

But Nginx's modules are lightweight, widely documented so that making it becomes more native in a way.

References

- [Installing NGINX Open Source](#)
- [Building nginx from Sources](#)
- [How To Set Up Nginx Server Blocks \(Virtual Hosts\) on Ubuntu 16.04](#)
- [Command-line parameters - nginx](#)

- [Launch Nginx on startup - ubuntu](#)
- [Debug Curl Requests](#)
- [HTTP headers - MDN Web Docs](#)
- [Configuring HTTPS servers - nginx](#)
- [How To Configure Nginx to use TLS 1.2 / 1.3 only](#)
- [Create a Self-Signed Certificate for Nginx in 5 Minutes](#)
- [Certbot instructions for NGINX](#)
- [Module ngx_http_geoip_module - nginx](#)
- [GeoIP | NGINX Documentation](#)